

A large, abstract yellow shape in the top-left corner of the slide, with a thin yellow line extending from its bottom edge towards the center.

JS

JavaScript

Les bases de la syntaxe

Vito Deriu

A large, abstract yellow shape in the bottom-right corner of the slide, with a thin yellow line extending from its top edge towards the center.

AIX
ynov
CAMPUS

Présentation

JavaScript est un langage de programmation interprété par le navigateur qui permet d'ajouter des fonctionnalités d'**interaction** grâce à l'utilisation de **script** pour créer des applications web plus **riches et dynamiques**.



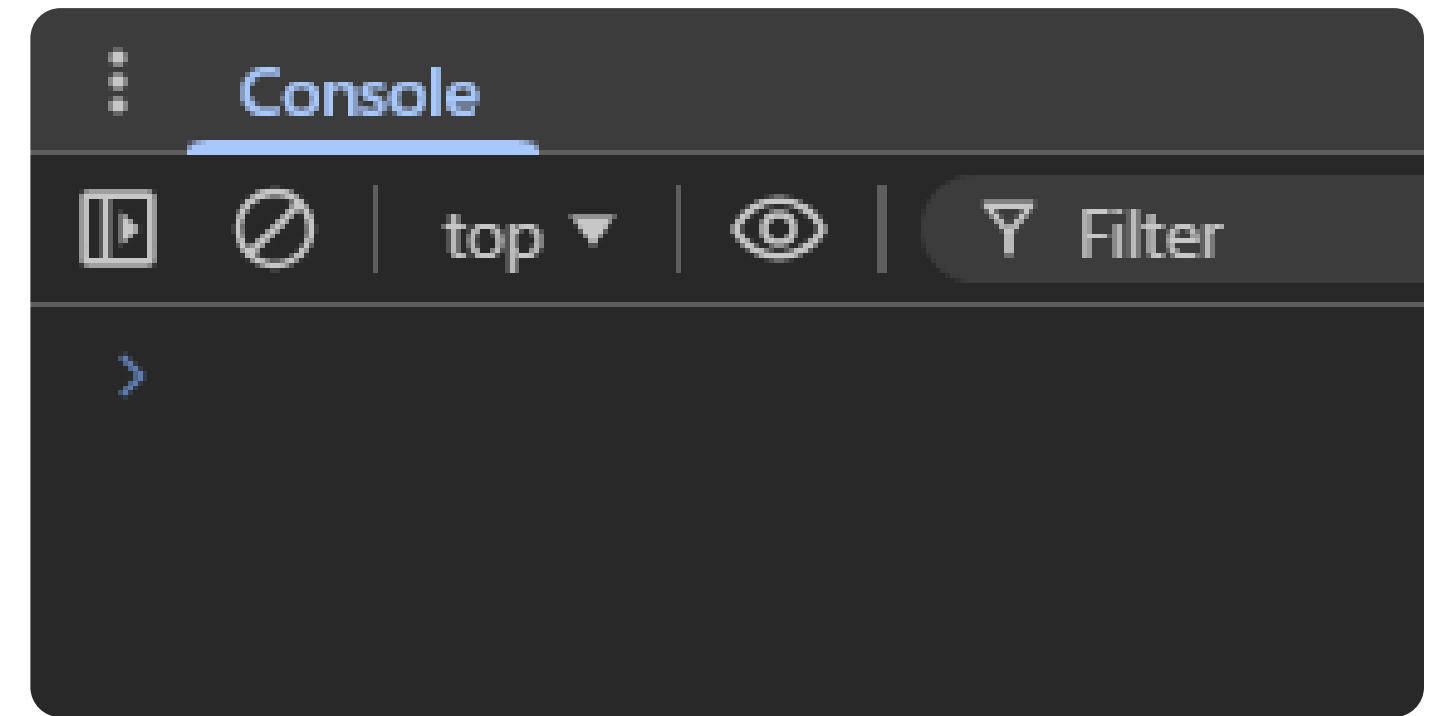
JavaScript est aussi utilisé en dehors des navigateurs grâce notamment à l'environnement **NodeJS**.

Attention à ne pas confondre Java et JavaScript, ce sont deux langage **différents** !



Présentation

Il est possible d'exécuter directement du JavaScript dans la console du navigateur (F12 ou clic droit → inspecter l'élément)



Testez par vous même en écrivant :

```
console.log("J'adore le JS !");
```

Types de données

JavaScript est un langage **faiblement typé** ce qui signifie qu'une variable peut prendre n'importe quel type au cours de son existence.

Un **tableau** peut également contenir plusieurs type de variable à l'intérieur.

Un **objet** est une liste de paire { clé: valeur }.

Chaîne de caractère	"Vito" 'Vito' `Vito`
Nombres	10, -2, 3.14
Booléen	true, false
Tableau	["Vito", "Cyril", 23, true]

Objet :

```
let mentor = {  
  name: "Vito",  
  age: 28,  
  hair: false  
}  
console.log(mentor)
```

Déclarer une variable

let

```
let myVar= 22
```

```
let myVar //undefined
```

```
let myVar = undefined
```

```
let myVar = 'Info'
```

```
myVar = 'Cyber'
```

const

```
const myConst = 'Vito'
```

var

```
var myVar = false
```

A noter :

La portée de la variable est différente entre let et var. la portée de let est limité au bloc dans lequel la variable est déclaré.

La portée de var se limite a sa fonction. On préférera utiliser let dans la plupart des cas.

Les opérateurs

Opérateurs d'affectation `=`

Opérateurs de calcul `/ % + * -`

Opérateurs de comparaison `=== == > < <= >= != !==`

Opérateurs de comparaison `&& || !`

```
x = 3  
x = 5 + y  
x += 2.45
```

```
x === y  
x !== y  
x < y
```

```
(x > y) && (x !== 0)
```

Les conditions

If - elseif - if

```
if (nbVerre <= 1 ){
    return "Tu peux y aller !"
} else if (nbVerre > 10){
    return "Tu es mort !"
} else {
    return "Stop ! Tu rentres à pied !"
}
```

Ternaire :

```
let glacon = boisson === "Ricard" ? "Oui" : "Non"
```

Switch - case

```
switch (filiere) {
    case 1:
        console.log("Cyber")
        break;
    case 2:
        console.log("IA Data")
        break;
    default:
        console.log("Informatique")
        break;
}
```

Les boucles

for

```
for (let i = 0; i < 10; i++) {  
    console.log(i)  
}
```

do ... while

```
let i = 0;  
do {  
    i++;  
    console.log(i);  
} while (i < 10);
```

while

```
let j = 0;  
while (j < 10) {  
    j++;  
    console.log(j);  
}
```

forEach

```
let nameList = ["Alan", "Vito", "Cyril"];  
nameList.forEach((name) => console.log(name));
```


Les fonctions

```
function myFunction(param1, param2) {  
    console.log(param1 + param2);  
}  
myFunction(1, 2);  
// résultat : 3
```

```
function testAlcool (nbVerre){  
    if (nbVerre <= 1 ){  
        return "Tu peux y aller !"  
    } else if (nbVerre > 10){  
        return "Tu es mort !"  
    } else {  
        return "Stop ! Tu rentres à pied !"  
    }  
}
```

```
let verre = 0  
console.log(testAlcool(verre))  
//Tu peux y aller !  
console.log(testAlcool(3))  
//Stop ! Tu rentres à pied !
```

```
function servir(boisson){  
    let glacon = boisson === "Ricard" ? "Oui" : "Non"  
    console.log("faut-il des glaçons : " + glacon)  
}
```

```
servir("Ricard")  
    //faut-il des glaçons : Oui  
servir("Tisane")  
    //faut-il des glaçons : Non
```

Les fonctions fléchées



Les fonctions fléchées sont une autre façon plus **courte** et **légère** d'écrire des fonctions. Elles sont très utiles dans les boucles **forEach** ou avec **map()** par exemple.

```
const arrowFunc = (a, b) => {  
  return a + b  
}  
arrowFunc(1, 2)
```

```
let nameList = ["Alan", "Vito", "Cyril"];  
nameList.forEach((name) => console.log(name));
```

Si la fonction comprends qu'un seul **return** on peut simplifier :

```
const ezArrowFunc = (a, b) => a + b  
ezArrowFunc(1, 2)
```

Si un seul paramètre,

Parenthèses facultatives

```
const oneParam = a => a * a  
oneParam(2)
```

Si aucun paramètre,

Parenthèses obligatoires

```
const noParam = () => "Hello World"  
noParam()
```

Les commentaires

commentaires

```
// Ceci est un commentaire sur une ligne
```

```
/* Ceci est un commentaire  
sur plusieurs lignes */
```

```
/**  
 * Ceci est un commentaire de documentation  
 * Additionne deux nombres.  
 * @param {number} a - Premier nombre  
 * @param {number} b - Deuxième nombre  
 * @returns {number} La somme de a et de b  
 */  
function add(a, b) {  
    return a + b;  
}
```

```
function add(  
    a: number,  
    b: number,  
): number
```

Ceci est un commentaire de documentation
Additionne deux nombres.

Params: a – Premier nombre
b – Deuxième nombre

Returns: La somme de a et de b